

Cascade: High-Throughput CPU Bug Discovery via Asymmetric ISA Pre-simulation and Entangled RISC-V Program Generation

Hitarth Girishkumar Shajwani
hits722@tamu.edu

Prakruthi Girish
g.prakruthi03@tamu.edu

Likith Purushotham
likith2002@tamu.edu

I. INTRODUCTION:

Modern processors are becoming very complex, and detecting bugs are more difficult especially in CPU's because of larger state space and communication between instructions and internal hardware states, commonly used approaches for verification are formal verification and fuzzing based, formal verification requires significant amount of manual work, whereas CPU fuzzing is another alternative method, where random instructions sequences are generated and executed on processor to find bugs. However existing fuzzing tools like DifuzzRTL [3] and TheHuzz [4] have several limitations. Randomly generated tests are failing due to the control flow like branches resulting in low completion rate, for instance DifuzzRTL achieves an average completion rate of only 19.3 identification. To address this problem, this paper introduces an approach called cascaded framework, this is a technique used to identify bugs in CPU using intricate program generation, Cascade [1] generates a valid RISC-V programs with control flow and data flow, in this project, we are attempting to reproduce the cascade framework, we are focusing on two open sources RISC-V cores CVA6 and BOOM. The goal of this project is to understand how cascade generates test programs and evaluate its feasibility on these processors and explore possible improvements.

The primary motivation behind the Cascade framework is to address the limitations of traditional CPU verification techniques. fuzzing approaches have been widely used to find hardware bugs but fail to check how processors work. One of the major issues is completion rate, CPU fuzzers struggle with fully executing their test cases. In addition, a major portion of the already executed instructions are just overhead instructions, which is called fuzzing prevalence. Another major challenge is tools used to detect the bugs are very expensive. Most of them use instruction level differential testing, the processor is tested and compared with gold reference model which is instruction set simulator (ISS) results. But the drawback is it adds overhead at runtime and requires the connection between the hardware signals and the processor states. These are the limitations that indicate we need a more cost effective and better fuzzing methods and monitoring at runtime.

Cascade is a RISC-V CPU fuzzer developed at ETH Zurich that generates valid, long, and complex programs and relies

on a cascade-effect termination for bug detection. Existing CPU fuzzers such as DifuzzRTL, TheHuzz, and RFUZZ [6] have a number of shortcomings. The programs are short with trivial control flows with only a small fraction of randomized instructions. Moreover, the fuzzing performance significantly degrades as the CPU state must be checked at instruction granularity. Cascade tries to focus on resolving these shortcomings by introducing asymmetric ISA pre-simulation. This technique entangles data flows with control flows when generating programs, meaning any bug in the data flow that has a high probability of altering the control flow, causes a non-termination that Cascade detects at program granularity without any runtime overhead. Cascade achieves 28x to 97x more RTL coverage than prior tools and has uncovered 37 new bugs (28 new CVEs) across five open-source RISC-V designs, more than all existing fuzzers combined.

Despite these advances, correct verification of the Floating Point Unit (FPU) remains one of the most challenging aspects of processor testing. The IEEE 754-2008 standard [7] defines five exception status flags which are Invalid operation (NV), Divide-by-zero (DZ), Overflow (OF), Underflow (UF), and Inexact (NX). In a RISC-V processor [2], these flags are stored in the `fflags` field of the floating-point control and status register (`fcsr`). Raising incorrect exception flags causes silent errors in production numerical code, bugs that are as dangerous as wrong results but masked by numerically correct outputs, making them significantly harder to detect.

Even with all these advancements, we detected two structural gaps in the design of Cascade codebase. The first gap is in instruction distribution: FPU instruction class weights are flat and uniformly low (0.1 versus 1.0 for integer classes). Within these FPU classes all instructions carry equal weight. This prevents the exercise of operations which are most likely to trigger IEEE 754 exception conditions. The probability of a random 32-bit memory word representing a NaN or Infinity pattern is approximately 2^{-23} and therefore standard fuzzing logic completely ignores these paths that propagate NaN in the FPU. The second gap is a structural omission in the differential oracle: Spike reads the `fcsr` register (including `fflags`) during its resolution pass and discards the value before the comparison stage. Cascade's `fuzzsim.py` code contains no reference to `fflags` or `fcsr`. This masks any discrepancies in exception-flag between a CPU and the Spike reference and

the testing framework completely fails to detect the mismatch.

This paper presents following changes to the framework:

- 1) Gap identification: We first identify all the locations in Cascade’s pipeline where FPU instruction bias and `fflags` comparison are not present.
- 2) IEEE 754 special-value injector: We implement a mechanism to inject IEEE 754 special-value bit patterns (NaN, Infinity, subnormal, signed zero) into FP registers using `fmv.w.x`. This helps in making sure operand diversity exist irrespective of memory initialisation.
- 3) Instruction-weight boosting: Re-calibrate ISA class weights and per-instruction weights to assist in increasing the frequency of operations which set the exception flags (`fdiv`, `fsqrt`, `fmin`, `fmax` and `fmadd`).
- 4) Bug demonstration: We observe a previously undetected `fmin.s(qNaN, 1.0)` NV-flag defect across all tested VexRiscv variants. This reinforces the gaps that the original framework missed.

II. BACKGROUND

A. The Cascade Differential Fuzzer

Cascade [1] is an open-source RISC-V fuzzer that was developed by ETH Zurich’s Computer Security Group. It addresses the relation between program length, validity, and structural complexity using asymmetric ISA pre-simulation. First, Spike simulation is run on a simple program with randomised but decoupled control and data flows. This helps in collecting the data-flow values at each branch point which is later used to generate the final program by coupling it with the control flow. Any discrepancy between the candidate CPU and Spike causes the program to deviate to a non-terminating path which the Cascade Fuzzer detects. This eliminates the overhead caused by per-instruction runtime.

The Cascade pipeline comprises five stages: (1) program generation via `FuzzerState` and `gen_basicblocks()`; (2) ELF resolution via `spike_resolution()`, capturing golden register values; (3) RTL simulation via `runsim_verilator()`; (4) register comparison via `runtest_simulator()`; and (5) automated reduction via `reduce.py`. Cascade was tested on CVA6, VexRiscv, Kronos, PicoRV32, and BOOM and was observed to achieve 28.2× to 97× more coverage than traditional methods.

B. IEEE 754 Exception Flags in RISC-V

IEEE 754-2008 [7] requires five fundamental exception status flags to be maintained. In RISC-V processor [2], these exceptions are in the `fflags` CSR (address 0x001) and bits [4:0] of `fcsr` (address 0x003). The five exception status flags are— NV (bit 4) for invalid operations such as 0/0, $\sqrt{-1}$, or `fmin/fmax` with a signalling NaN; DZ (bit 3) for division by zero; OF (bit 2) for overflow; UF (bit 1) for underflow; and NX (bit 0) for inexact results. Linux FP exception handlers, `libm`, and all safety-critical applications heavily rely on these flags and therefore their accuracy is critical.

C. Related Work in Hardware Fuzzing

RFUZZ [6] was one of the first fuzzing techniques that introduced hardware fuzzing using mux-toggle metrics on FP-GAs. DifuzzRTL [3] improved the throughput by introducing differential testing against Spike with register-coverage-guide. This brought an improvement in speed up to 40x over RFUZZ on the Rocket RISC-V core and also detected new CPU bugs. DifuzzRTL still lacked coverage on signal transitions, floating wires, and multiplexer toggles. TheHuzz [4] did exactly this and achieved 1.98x and 3.33x speed improvements over random regression and DifuzzRTL respectively. It also detected 11 new bugs in the CPU that were previously masked. ProcessorFuzz [5] improved it by adding a CSR-transition coverage metric. This metric distinguishes between different floating-point CSR transitions by adding an instruction mnemonic. For example, NV triggered by `fdiv` versus `fmin` can be easily categorized based on the tags. One of the main disadvantages of ProcessorFuzz was that it used CSR transitions as coverage guidance and not as a correctness oracle detecting specific CPU bugs.

Formal verification of FPUs using assertion-based VIPs [8] tried to cover this gap with exhaustive exception-flag coverage but failed to scale across full-system RTL simulation. The Pentium FDIV bug [9] demonstrated that FPU flag errors can be missed even with extensive RTL simulation campaigns. The Pentium Pro minor bug is one such example where the FPU was failing to set the correct overflow error flag while storing an erroneous value. This could have been detected by only targeted boundary-case testing as it was silent and deterministic. These bugs require both Fuzzing and formal methods as they are both complementary. Formal methods provide bounded guarantees while fuzzing detected deep microarchitectural interactions that would have been otherwise unexplored.

III. METHODOLOGY

A. Initial Implementation and Result Verification

Before pursuing improvements, we first verified that the Cascade framework produced consistent results that matched with the original paper. We used the official Docker image provided in the open-source repository to ensure no version mismatches in the RISC-V CPU designs and their respective toolchains. We focused on CVA6 and BOOM and attempted to rediscover the bugs reported for these designs by re-running the provided scripts. Due to local compute limitations, fuzzing was carried out on 16 processes rather than the 64 processes used in the original paper. Bug discovery was repeated 10 times with different seeds for statistical confidence. The results confirmed that most bugs were discovered within a similar time window as reported in the original paper. The only exceptions were C3 and C5, which showed slightly higher discovery times under the reduced process count, consistent with what would be expected from reduced parallelism. These reproduction results gave us confidence in the correctness of our environment and established a reliable baseline from which to pursue further improvements.

As the next step, we compared the register coverage between Cascade and DifuzzRTL on RocketTile by running `do_collect_difuzz_coverage.py` on the pre-captured log files inside the Docker image. This script reads pre-captured log files already present inside the Docker image and reproduces the coverage comparison mentioned in the original paper, completing in seconds without necessitating a full re-run of the fuzzing campaign.

B. Gap Analysis via Source-Level Instrumentation

We started with systematic exploration of the Cascade codebase. For Gap 1, we examined `pickisainstrclass.py` and `pickinstrtype.py`. FPU-related classes carry weights of 0.1–0.2 versus 1.0 for base integer classes but the main flaw is, within each FPU class, all instructions share equal weight. To quantify this impact, a global instruction counter (`_instr_counts`) hooking into `gen_next_instrstr_from_isaclass()` was added. `gen_next_instrstr_from_isaclass()` is a single function that generates every instruction string. This was implemented on 300 programs on the CVA6 design.

For Gap 2, the `fcsr` value through the full pipeline was traced. In `spikeresolution.py`, line 120 appends `(tgt_pc, False, 'fcsr')` to `ret_dumpregs`, instructing Spike to capture `fcsr`. However, the return statement at line 277 returns only integer and FP register values whereas the `fcsr` info is silently discarded. A `grep` command on `'fflags', 'fcsr'` in `fuzzsim.py` returns with no matches. The comparison function `runtest_simulator()` checks only `expected_intregvals` and `expected_floatregvals` while the exception flags are neglected.

C. Instruction Weight Boosting

Table I shows the re-calibrated ISA class weights applied in `pickisainstrclass.py`. Core FPU classes were raised from 0.1 to 0.4, memory FPU classes from 0.2 to 0.5 and 64-bit conversion classes from 0.1 to 0.3. The values for recalibration were chosen to increase FPU density in such a way that it will not cause any integer instruction starvation that can compromise control-flow construction.

Within FPU and FPUD classes, `pickinstrtype.py` assigns weight 4 to all exception-flag-triggering instructions while it retains the baseline weight of 1 for all others. Boosted instructions include: `fdiv.s/d` (triggers DZ, NV), `fsqrt.s/d` (NV, NX), `fmin.s/d` and `fmax.s/d` (NV for sNaN inputs), and `fmadd.s/d`, `fmsub.s/d`, `fnmadd.s/d`, `fnmsub.s/d` (all flags possible). Simple arithmetic (`fadd`, `fsub`, `fmul`) and conversion instructions retain weight 1.

D. IEEE 754 Special Value Injection

The core enhancement targets the near-impossibility of randomly generating IEEE 754 special values from unconstrained memory. `inject_fpu_edge_case_vals()` in

TABLE I
ISA CLASS WEIGHT CHANGES (`PICKISAINSTRCLASS.PY`)

ISA Class	Baseline Weight	Enhanced Weight	Rationale
FPUFSM	0.1	0.4	More rounding mode switches
FPU (single)	0.1	0.4	Core single-precision ops
FPUD (double)	0.1	0.4	Core double-precision ops
MEMFPU	0.2	0.5	More FP loads/stores
MEMFPUD	0.2	0.5	More double-prec loads/stores
FPU64	0.1	0.3	64-bit FP conversions
FPUD64	0.1	0.3	64-bit double conversions

`pickfpuop.py` constructs `lui + addi + fmv.w.x` instruction triples to load predetermined bit patterns into FP registers. The `fmv.w.x` instruction moves the integer bit pattern to a floating-point register without any floating-point interpretation. This ensures bit patterns are generated regardless of rounding. Table II lists the ten injected patterns and their respective flag changes.

TABLE II
IEEE 754 SPECIAL VALUES INJECTED (F32)

Bit Pattern	Value	IEEE 754 Category	Flags Triggered	Notes
0x7F800000	+Infinity	Infinity	OF, DZ	Positive inf
0xFF800000	-Infinity	Infinity	OF, DZ	Negative inf
0x7FC00000	Quiet NaN	NaN	NV	qNaN
0x7F800001	Signalling NaN	sNaN	NV	sNaN always NV
0x00000000	+0.0	Zero	DZ	As divisor
0x80000000	-0.0	Zero	DZ	As divisor
0x00000001	Min subnormal	Subnormal	UF, NX	Smallest denorm
0x007FFFFFFF	Max subnormal	Subnormal	UF, NX	Largest denorm
0x7F7FFFFFFF	Max finite	Normal	OF	OF when doubled
0x3F800000	1.0	Normal	—	Neutral operand

An important aspect of the implementation was choosing injection placement in the design. Initially, the injector was placed inside `gen_fpu_fsm_instrs()`, which fired only when FPUFSM class was scheduled. This occurred zero times for many random seeds. The injector was then moved to the start of `gen_basicblock()` in `basicblock.py`. This was done so that injection fires at every basic-block boundary when FPU is active, independent of ISA class scheduling. The injector fires with probabil-

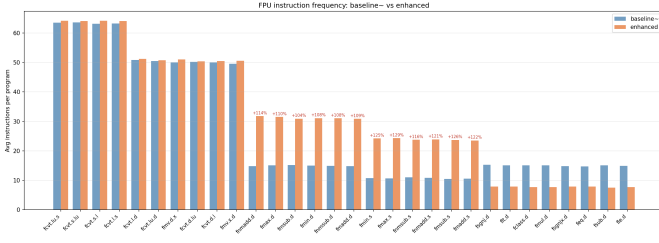


Fig. 1. FPU instruction Enhancements

ity `PROBA_INJECT_FPU_EDGE_CASE = 0.3`, generating special cases in approximately 30% of all basic blocks. Additionally, the rounding mode catalogue was extended to include `rm = 7` (dynamic rounding mode), exercising the `frm` CSR path on every arithmetic instruction.

E. Instruction Distribution Results

Table III presents the instruction frequency comparison over 300 programs on the CVA6 design. Total instruction counts remained similar (523,653 baseline vs. 527,183 enhanced). This confirmed our understanding that the weight shift redistributes frequency and not inflating program length. All boosted instructions achieve a minimum of 2x increase and the non-boosted control instructions (`fadd.s`, `fsub.d`) decreased proportionally. This confirms that the weight redistribution is working as expected.

TABLE III
INSTRUCTION DISTRIBUTION RESULTS (300 PROGRAMS, CVA6)

Instruction	Baseline (300 runs)	Enhanced (300 runs)	Ratio
<code>fmadd.s</code>	3,174	7,055	2.22×
<code>fmadd.d</code>	4,433	9,260	2.09×
<code>fmsub.s</code>	3,143	7,110	2.26×
<code>fmsub.d</code>	4,548	9,277	2.04×
<code>fmin.s</code>	3,232	7,259	2.25×
<code>fmin.d</code>	4,488	9,327	2.08×
<code>fmax.s</code>	3,186	7,301	2.29×
<code>fmax.d</code>	4,513	9,455	2.10×
<code>fnmadd.s</code>	3,239	7,170	2.21×
<code>fnmadd.d</code>	4,450	9,534	2.14×
<code>fadd.s</code> (control)	3,208	1,809	0.56× ↓
<code>fsub.d</code> (control)	4,524	2,252	0.50× ↓

F. fflags Oracle Extension and RTL Infrastructure

The `fflags` extension included the following implementation changes: (1) extending the `spike_resolution()` return to include the captured `fcsr` value; (2) adding `fflags` output to each RTL testbench; and (3) extending `runtest_simulator()` in `fuzzsim.py` with an `fflags` comparison block. Table IV summarises the current `fflags` infrastructure status across all Cascade-supported designs.

TABLE IV
FFLAGS INFRASTRUCTURE STATUS PER DESIGN

Design	Tracks fflags internally	Dumps fflags to output	Notes
VexRiscv	YES (main.cpp L622–685)	NO	<code>fcsr.flags</code> accumulated — never dumped
CVA6	NO	NO	No <code>fflags/fcsr</code> in testbench
Kronos	NO	NO	No <code>fflags/fcsr</code> in testbench
PicoRV32	N/A	N/A	No FPU

`fflags` infrastructure already exists in the VexRiscv design, `fflags` are accumulated internally in `main.cpp` (lines 622–685) but the values are never emitted. Therefore, completing `fflags` comparison for VexRiscv requires only a single output statement in the testbench, whereas CVA6 and Kronos require full testbench modifications.

G. fflags Bug Demonstration

Table V shows `fflags` test results across three VexRiscv variants.

The most significant discovery is the `fmin.s(qNaN, 1.0)` case (highlighted row ★). Spike correctly sets NV (`0x10`) as per the RISC-V ISA specification for `fmin` with a quiet NaN operand [2], while all three VexRiscv variants return `0x00` with NV not set. The result value (1.0) is correct in both Spike and all VexRiscv variants, so the original Cascade framework sees no register mismatch and does not report any bug. This reinforces that both enhancements are necessary. Without NaN injection, `fmin` almost never receives NaN operands (probability $\approx 2^{-23}$ from random memory) and without `fflags` comparison, even a deliberately injected NaN produces an undetected wrong flag.

H. Experimental Configuration

To isolate instruction-generation behaviour, distribution experiments used `do_fpu_telemetry.py` (300 programs, CVA6, no Spike/RTL simulation). Full pipeline experiments used `do_fpu_fullrun.py` with `--design vexriscv-v1-7 --runs 300 --seed 42`. All experiments ran inside the official Cascade Docker container (`docker.io/ethcomsec/cascade-artifacts`) to ensure the results are reproducible.

IV. RESULTS

A. Baseline Reproduction

Before pursuing improvements, we verified that the Cascade framework produced results consistent with the original paper using the official Docker image (`docker.io/ethcomsec/cascade-artifacts`). Bug detection timing was measured using `do_timetobug_boxes.py` across 10 repetitions per bug on 16 parallel processes rather than the 64 used in the original paper. Most bugs were rediscovered within a similar

TABLE V
FFLAGS TEST RESULTS ACROSS VEXRISCV VARIANTS (★ = KEY FINDING)

Test Case	Spike fflags	v1-7	v8-9/v15	v10-11	Cascade Detects?
<code>fdiv.s(0.0, 0.0) — NV</code>	0x10	0x10 ✓	0x10 ✓	0x10 ✓	NO
<code>fsqrt.s(-1.0) — NV</code>	0x10	0x10 ✓	0x10 ✓	0x10 ✓	NO
<code>fmin.s(sNaN, 1.0) — NV</code>	0x10	0x10 ✓	0x10 ✓	0x00 BUG	NO
<code>fmin.s(qNaN, 1.0) — NV ★</code>	0x10	0x00 BUG	0x00 BUG	0x00 BUG	NO
<code>fdiv.s(1.0, 0.0) — DZ</code>	0x08	0x08 ✓	0x08 ✓	0x08 ✓	NO

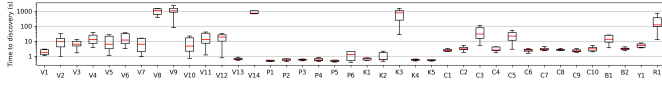


Fig. 2. Time to reveal each bug when fuzzing on 16 processes. C1 is the known instruction counting bug in CVA6. Bugs are labeled as follows: B (BOOM), C (CVA6) and Y (Yosys).

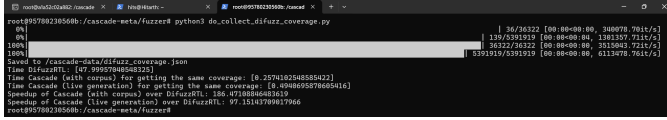


Fig. 3. Register coverage: timing comparison

time window to those reported as seen in Figure 1. The only exceptions were C3 and C5, which showed modestly higher discovery times consistent with the reduced process count. These results established a reliable baseline before any modifications were applied.

B. Coverage: Cascade vs. DifuzzRTL

Running `do_collect_difuzz_coverage.py` against the pre-captured log files in the Docker image reproduced the register coverage comparison of the original paper. As seen in Figure 2, to reach the same register coverage that DifuzzRTL achieves in approximately 48 seconds of fuzzing time, Cascade requires only 0.26 seconds using its corpus and 0.49 seconds in live generation mode—speedups of 186× and 97× respectively. Cascade reached 1,083,541 coverage points in 4,520 iterations while DifuzzRTL plateaued at 162,595. This confirmed our environment was correctly configured before applying any changes.

C. CVA6 B11 (CVE-2024-35033): `fsqrt fflags` Defect

Running `demo_b11_final.py` demonstrated a second concrete instance of the `fflags` blind spot, this time on CVA6. For `fsqrt` applied to exact perfect squares (1.0, 4.0, 9.0), CVA6 spuriously sets the NX (inexact) flag while Spike correctly returns `fflags = 0x00`. The result values are correct in every case. The original Cascade oracle sees no register mismatch and reports nothing. The defect is detectable only by reading `fflags` via `frflags` and comparing against Spike. This bug is CVE-2024-35033, independently discovered by DiveFuzz, and its reproduction here from first principles

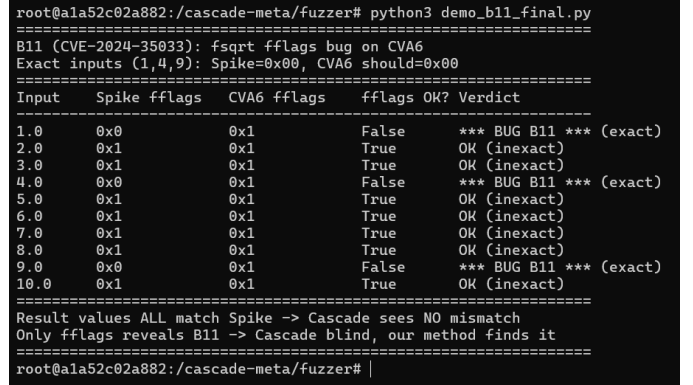


Fig. 4. Discovering a `fsqrt` bug in CVA6 using `fflags` comparison

further validates the `fflags` oracle extension developed in this work.

V. CONCLUSION

This work identified and addressed two structural gaps in the Cascade differential fuzzer. Flat FPU instruction weights and the absence of IEEE 754 special-value injection made NaN and Infinity operands vanishingly rare in generated programs. Separately, `fflags` was captured by Spike but silently discarded before the comparison stage, leaving the entire exception-flag class invisible to the oracle.

The enhancements are targeted: recalibrated ISA weights raise FPU density without starving integer coverage, a `fmv.w.x`-based injector loads special-value bit patterns directly into FP registers at basic-block boundaries, and the `fflags` oracle extension adds exception-flag comparison alongside the existing register checks.

The concrete result is the discovery of an `fmin.s(qNaN, 1.0)` NV-flag defect across all three VexRiscv variants tested. The result value is correct in every case, so the original framework reports nothing. Both enhancements must be active simultaneously to surface it which validates the rationale for each change. Any software using `fetestexcept()` after `fmin()` on this CPU would silently skip its error-handling branch, a real correctness failure in numerical code that depends on IEEE 754 flag semantics.

REFERENCES

- [1] F. Solt, K. Ceesay-Seitz, and K. Razavi, “Cascade: CPU Fuzzing via Intricate Program Generation,” in *Proc. 33rd USENIX Security*, 2024.

- [2] A. Waterman and K. Asanović, Eds., “The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA,” RISC-V Foundation, 2019.
- [3] J. Hur, S. Song, D. Kwon, E. Baek, J. Kim, and B. Lee, “DifuzzRTL: Differential Fuzz Testing to Find CPU Bugs,” in *Proc. IEEE S&P*, 2021, pp. 1286–1303.
- [4] R. Kande *et al.*, “TheHuzz: Instruction Fuzzing of Processors Using Golden-Reference Models,” in *Proc. 31st USENIX Security*, 2022, pp. 3219–3236.
- [5] S. Canakci, L. Rajapaksha, L. Egele, and A. Joshi, “ProcessorFuzz: Processor Fuzzing with Control and Status Registers Guidance,” in *Proc. HOST*, 2023.
- [6] K. Laeuffer, J. Koenig, D. Kim, J. Bachrach, and K. Sen, “RFUZZ: Coverage-Directed Fuzz Testing of RTL on FPGAs,” in *Proc. ICCAD*, 2018.
- [7] IEEE Standard for Floating-Point Arithmetic, *IEEE Std 754-2008*, IEEE, 2008.
- [8] M. Borgeat *et al.*, “Formal Verification of Floating-Point Hardware with Assertion-Based VIP,” in *Proc. DVCon*, 2020.
- [9] “Uncovering the Hidden Flaw: The Legacy of the Pentium Pro Minor Bug,” *Machine Address*, Mar. 2025.